



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA



ESTRUCTURAS DE DATOS Y ALGORITMOS I

TRABAJO FINAL

SIMULADOR DE UN SISTEMA OPERATIVO

REPORTE TÉCNICO

Integrantes

- Montiel Martínez Pamela
- Padilla Pérez Valeria
- Toledo Valencia Aimée Guadalupe

Profesora: Ing. Adara Mercado Martínez

Grupo: 5

Semestre: 2026-2

Fecha de entrega: 10 de junio de 2026

Índice

1. Introducción
2. Objetivos
3. Planteamiento del problema
4. Marco teórico
5. Descripción general del proyecto
6. Arquitectura y organización del sistema
7. Desarrollo e implementación de estructuras de datos
8. Sistema de planificación de procesos
9. Administrador dinámico de memoria
10. Integración entre módulos
11. Benchmarking y análisis experimental
12. Sistema de pruebas y validación
13. Problemas encontrados y soluciones implementadas
14. Complejidad algorítmica
15. Decisiones de diseño e ingeniería
16. Resultados obtenidos
17. Aprendizajes adquiridos
18. Mejoras futuras
19. Bitácora de uso de la IA
20. Conclusiones
21. Referencias

1. Introducción

Dentro de la Ingeniería en Computación, las estructuras de datos representan uno de los pilares fundamentales para el desarrollo de software eficiente, escalable y organizado. Gran parte de los sistemas computacionales modernos dependen directamente de cómo se almacena, administra y manipula la información en memoria.

Durante el desarrollo de la materia Estructuras de Datos y Algoritmos I se abordaron conceptos fundamentales relacionados con:

- memoria dinámica
- estructuras lineales
- listas enlazadas
- complejidad algorítmica
- modularidad
- abstracción de datos
- administración eficiente de recursos

Sin embargo, comprender verdaderamente estos conceptos requiere ir más allá de ejercicios pequeños aislados. Por esta razón, se desarrolló el presente proyecto integrador, cuyo propósito fue aplicar de manera conjunta los conocimientos adquiridos durante el semestre mediante la construcción de un sistema modular inspirado en algunos mecanismos fundamentales utilizados por los sistemas operativos.

El proyecto consiste en un simulador de planificación de procesos y administración dinámica de memoria desarrollado principalmente en lenguaje C, complementado con herramientas de análisis y benchmarking implementadas en Python.

La elección del lenguaje C fue especialmente importante debido a que este permite trabajar directamente con:

- punteros
- memoria dinámica
- estructuras enlazadas
- direcciones de memoria
- administración manual de recursos

Esto permitió comprender de forma mucho más profunda el funcionamiento interno de las estructuras de datos.

El proyecto incorpora:

- pilas
- colas
- colas circulares
- algoritmos de planificación
- administración dinámica de memoria
- benchmarking
- automatización

- pruebas funcionales

Además, el sistema fue organizado mediante una arquitectura modular dividida en múltiples componentes independientes, permitiendo una separación clara de responsabilidades y un diseño mucho más cercano a proyectos reales de ingeniería de software.

Uno de los aspectos más importantes del proyecto fue lograr que todas las estructuras interactuaran entre sí dentro de un mismo sistema funcional y coherente, simulando parcialmente la manera en que distintos módulos colaboran dentro de sistemas computacionales reales.

Finalmente, este proyecto representó una experiencia práctica sumamente importante para comprender cómo las estructuras de datos constituyen la base sobre la cual se construyen sistemas complejos dentro del área de la computación.

2. Objetivos

Objetivo general

Desarrollar un sistema modular inspirado en componentes fundamentales de un sistema operativo utilizando estructuras de datos dinámicas, algoritmos de planificación y administración manual de memoria.

Objetivos específicos

- Implementar estructuras dinámicas utilizando memoria reservada en tiempo de ejecución.
- Aplicar correctamente el uso de punteros y referencias.
- Desarrollar módulos reutilizables mediante abstracción de datos.
- Simular algoritmos de planificación de procesos.
- Implementar un administrador dinámico de memoria.
- Reducir fragmentación mediante coalescencia.
- Analizar complejidad temporal de operaciones fundamentales.
- Automatizar pruebas funcionales.
- Integrar Python para benchmarking y análisis experimental.
- Relacionar teoría de estructuras de datos con aplicaciones reales.

3. Planteamiento del problema

Uno de los principales desafíos dentro de la computación consiste en administrar eficientemente los recursos disponibles de un sistema.

En sistemas operativos reales, el procesador debe decidir constantemente:

- qué proceso ejecutar
- cuánto tiempo asignarle
- cuándo cambiar de tarea
- cómo administrar memoria disponible

De manera similar, las estructuras de datos deben permitir:

- acceso eficiente
- inserciones rápidas
- reutilización de memoria
- organización adecuada de información

El problema principal que motivó este proyecto fue desarrollar un sistema capaz de:

- administrar procesos
- reutilizar memoria
- organizar estructuras dinámicas
- mantener eficiencia operativa

Todo esto debía realizarse utilizando únicamente herramientas fundamentales vistas en la materia:

- listas enlazadas
- pilas
- colas
- punteros
- memoria dinámica

Además, el proyecto debía incorporar:

- modularidad
- pruebas
- benchmarking
- validación experimental

4. Marco teórico

4.1 Sistema Operativo

Un sistema operativo es un conjunto de programas encargados de administrar los recursos de una computadora y proporcionar servicios a los programas de usuario. Entre sus funciones principales se encuentran:

- Administración de procesos
- Gestión de memoria
- Asignación de recursos
- Control de dispositivos
- Planificación de la ejecución

En este proyecto únicamente se simulan algunos de estos mecanismos, con el fin de comprender su funcionamiento.

4.2 Procesos

Un proceso es una instancia de un programa en ejecución. Cada proceso requiere tiempo de CPU y memoria para completar sus tareas.

Dentro del simulador, cada proceso posee la siguiente información:

- Identificador único (PID)
- Tiempo total de ejecución
- Tiempo restante
- Prioridad
- Memoria requerida
- Estado del proceso

Los estados que puede presentar un proceso son:

- **READY**

Indica que el proceso está listo para ejecutarse.

- **RUNNING**

Representa al proceso que se encuentra utilizando el procesador.

- **BLOCKED**

Se refiere a los procesos que están esperando algún recurso.

- **FINISHED**

Corresponde a los procesos que ya han concluido la ejecución.

4.3 Scheduler

El scheduler es el componente encargado de determinar cuál proceso será ejecutado por el procesador.

Su principal objetivo es optimizar el rendimiento y distribuir adecuadamente el tiempo de CPU entre los diferentes procesos.

4.4 Algoritmo FIFO

FIFO (First In First Out) ejecuta los procesos en el mismo orden en que llegan al sistema.

Características:

- Fácil implementación
- Baja complejidad
- No considera prioridades
- Puede provocar largos tiempos de espera

4.5 Algoritmo Round Robin

Round Robin asigna una cantidad fija de tiempo denominada quantum a cada proceso.

Si el proceso no termina dentro del tiempo asignado, regresa al final de la cola para esperar nuevamente su turno.

Este algoritmo permite compartir el procesador de forma más equitativa entre todos los procesos.

4.6 Algoritmo Shortest Job First

Shortest Job First selecciona primero el proceso con menor tiempo de ejecución.

Su principal ventaja es reducir el tiempo promedio de espera y mejorar la eficiencia del sistema.

4.7 Estructuras de datos

Las estructuras de datos son mecanismos especializados utilizados para organizar información de manera eficiente dentro de memoria.

La elección de una estructura adecuada impacta directamente sobre:

- rendimiento
- complejidad
- consumo de memoria
- escalabilidad

En este proyecto se utilizaron principalmente estructuras dinámicas debido a que permiten reservar memoria durante ejecución.

4.8 Tipos Abstractos de Datos

Los Tipos Abstractos de Datos permiten separar:

- comportamiento
- implementación

Este principio fue aplicado mediante:

- archivos .h
- archivos .c

Esto mejora:

- modularidad
- mantenibilidad
- organización
- reutilización

4.9 Memoria dinámica

La memoria dinámica permite reservar memoria durante ejecución mediante funciones como:

- malloc()
- free()

El proyecto utiliza memoria dinámica en prácticamente todos sus módulos.

Esto permitió:

- crecimiento flexible
- reutilización de memoria
- construcción de estructuras enlazadas

4.10 Punteros

Los punteros almacenan direcciones de memoria.

Son fundamentales para:

- listas enlazadas
- estructuras dinámicas
- administración de memoria
- manipulación eficiente de datos

Gran parte de la complejidad del proyecto estuvo relacionada con:

- actualización correcta de referencias
- liberación segura
- validación de accesos

4.11 Pilas

La pila sigue el principio:

Last In, First Out (LIFO)

La estructura implementada utiliza listas enlazadas dinámicas.

Las operaciones principales fueron:

- Push
- Pop
- Peek

Complejidad:

- Inserción: $O(1)$
- Eliminación: $O(1)$

4.12 Colas

Las colas siguen el principio FIFO (First In First Out).

Se utilizan para administrar los procesos del scheduler FIFO.

Operaciones:

- Enqueue
- Dequeue
- IsEmpty

Complejidad:

- Inserción: $O(1)$
- Eliminación: $O(1)$

4.13 Colas circulares

Las colas circulares reutilizan posiciones liberadas mediante aritmética modular.

$$(i + 1) \bmod n$$

Esta estructura fue utilizada para implementar Round Robin.

Complejidad:

- Inserción: $O(1)$
- Eliminación: $O(1)$

4.14 Lista Ligada

Se utiliza para almacenar los procesos finalizados ordenados por PID.

Ventajas:

- Memoria dinámica
- Facilidad de inserción
- Flexibilidad

Complejidad:

- Recorrido: $O(n)$

4.15 Lista Doblemente Ligada

Se utiliza para representar los bloques de memoria y facilitar la coalescencia.

Permite desplazarse en ambos sentidos de la lista.

Complejidad:

- Inserción: $O(1)$
- Eliminación: $O(1)$
- Recorrido: $O(n)$

4.16 Algoritmos de planificación

La planificación de procesos consiste en decidir:

- qué proceso ejecutar
- cuándo
- y durante cuánto tiempo

En el proyecto se implementaron conceptos relacionados con:

- FIFO
- Round Robin

4.17 Algoritmos Implementados

Fuerza Bruta

Se utiliza para buscar bloques disponibles de memoria recorriendo toda la estructura.

Complejidad temporal:

$O(n)$

Algoritmos Greedy

Se implementaron:

First Fit

Selecciona el primer bloque libre que tenga suficiente espacio.

Best Fit

Busca el bloque más pequeño capaz de satisfacer la solicitud.

Worst Fit

Selecciona el bloque libre más grande.

Shortest Job First

Selecciona el proceso con menor tiempo de ejecución.

Divide y Vencerás

Esta técnica se utiliza para reorganizar la memoria y optimizar la administración de los bloques disponibles.

Backtracking

Permite recuperar estados anteriores y explorar soluciones válidas en caso de ser necesario.

4.18 Administración de memoria

La memoria se representa mediante bloques dinámicos enlazados entre sí.

Cada bloque almacena:

- Dirección inicial
- Tamaño del bloque
- Estado libre u ocupado
- PID asociado

Las principales operaciones realizadas son:

- Asignación de memoria
- Liberación de memoria
- Detección de fragmentación
- Unión de bloques libres

4.19 Fragmentación

La fragmentación ocurre cuando la memoria libre queda dividida en múltiples bloques pequeños dispersos.

Esto dificulta reutilización eficiente.

4.20 First Fit

First Fit busca el primer bloque libre suficientemente grande para satisfacer una solicitud.

Este algoritmo prioriza:

- simplicidad
- velocidad
- bajo costo computacional

4.21 Coalescencia

La coalescencia une bloques libres contiguos para reducir fragmentación externa.

Este proceso fue implementado automáticamente después de liberar memoria.

4.22 Complejidad algorítmica

La complejidad algorítmica mide el costo computacional de una operación respecto al tamaño de entrada.

En el proyecto predominan:

- complejidades $O(1)$
- complejidades $O(n)$

4.23 Benchmarking

El benchmarking consiste en medir experimentalmente el comportamiento de un sistema computacional.

En el proyecto se utilizó Python para:

- automatizar mediciones
- generar CSV
- construir gráficas.

5. Descripción general del proyecto

El proyecto consiste en un simulador modular inspirado en algunos componentes fundamentales utilizados por sistemas operativos.

El sistema integra:

- estructuras dinámicas
- planificación de procesos
- administración de memoria
- pruebas automatizadas
- benchmarking

La arquitectura fue organizada en módulos independientes.

Los principales componentes implementados fueron:

Módulo	Función
Stack	Administración LIFO
Queue	Administración FIFO

Módulo	Función
Circular Queue	Rotación de procesos
Scheduler	Planificación
Memory Manager	Gestión dinámica de memoria
Tests	Validación
Scripts Python	Benchmarking

6. Arquitectura y organización del sistema

El proyecto fue organizado utilizando una arquitectura modular.

Estructura principal

Carpeta	Función
include/	Interfaces
src/	Implementación
tests/	Validación
scripts/	Automatización
reports/	Resultados
docs/	Documentación

Esta organización permitió:

- separación de responsabilidades
- mantenimiento sencillo
- reutilización
- y mejor legibilidad

7. Desarrollo e implementación de estructuras de datos

7.1 Stack

La pila fue implementada utilizando listas enlazadas dinámicas.

Cada nodo contiene:

- un valor
- un puntero hacia el siguiente nodo

La pila mantiene únicamente referencia al elemento superior.

Las operaciones:

- push
- pop
- peek

presentan complejidad $O(1)$.

El uso de listas enlazadas permitió evitar tamaños fijos.

7.2 Queue

La cola utiliza:

- front
- rear.

Esto permite:

- inserciones eficientes,
- extracciones eficientes,
- y comportamiento FIFO correcto.

La implementación fue posteriormente reutilizada por el scheduler FIFO.

7.3 Circular Queue

La cola circular implementa:

- reutilización de posiciones
- rotación continua
- aritmética modular

Internamente administra:

- head

- tail
- size
- capacity

La utilización de aritmética modular permitió mantener complejidad constante sin desplazar elementos físicamente.

8. Sistema de planificación de procesos

8.1 Scheduler FIFO

El scheduler FIFO ejecuta procesos respetando orden de llegada.

El primer proceso en entrar será el primero en ejecutarse.

La implementación reutiliza Queue como estructura base.

Esto permitió separar:

- política de planificación
- almacenamiento interno

8.2 Round Robin

Round Robin distribuye tiempo de ejecución de forma rotativa.

La implementación mediante cola circular permitió:

- reutilización eficiente
- rotación continua
- simulación parcial de fairness

Aunque simplificado, el algoritmo conserva principios fundamentales utilizados por sistemas operativos reales.

9. Administrador dinámico de memoria

Uno de los componentes más importantes del proyecto fue el administrador de memoria.

Este módulo fue desarrollado utilizando listas doblemente enlazadas.

Cada bloque contiene:

- dirección inicial
- tamaño

- estado
- PID
- referencias siguientes
- referencias anteriores

9.1 Creación de memoria

`mm_create()` inicializa:

- el administrador
- el bloque inicial
- memoria total disponible

Inicialmente toda la memoria se encuentra libre.

9.2 First Fit

First Fit recorre secuencialmente los bloques buscando el primero suficientemente grande.

Cuando un bloque es mayor al tamaño solicitado:

- se divide
- se genera un nuevo bloque libre
- se conserva memoria restante reutilizable.

Esta implementación permitió simular fragmentación dinámica real.

9.3 Liberación de memoria

`mm_free()` permite liberar memoria utilizando:

- PID
- o dirección inicial

Después de liberar, el sistema ejecuta automáticamente coalescencia.

9.4 Coalescencia

La coalescencia une bloques libres contiguos.

Esto ayuda a:

- reducir fragmentación externa
- reutilizar memoria
- mantener estabilidad estructural

La implementación requirió actualización cuidadosa de referencias dobles.

10. Integración entre módulos

Uno de los aspectos más importantes del proyecto fue la integración real entre estructuras.

El sistema no fue construido como ejercicios independientes.

Por ejemplo:

- FIFO reutiliza Queue
- Round Robin reutiliza Circular Queue
- Memory Manager reutiliza listas dobles
- Tests validan múltiples módulos simultáneamente

Esto permitió construir un sistema mucho más cercano a proyectos reales.

11. Benchmarking y análisis experimental

El benchmarking fue desarrollado utilizando Python y librerías:

- pandas
- matplotlib
- numpy

El flujo general fue:

1. ejecutar programa
2. medir tiempos
3. exportar CSV
4. generar gráficas

Los resultados mostraron estabilidad temporal relativamente constante.

Esto coincide con:

- complejidades eficientes
- tamaños moderados

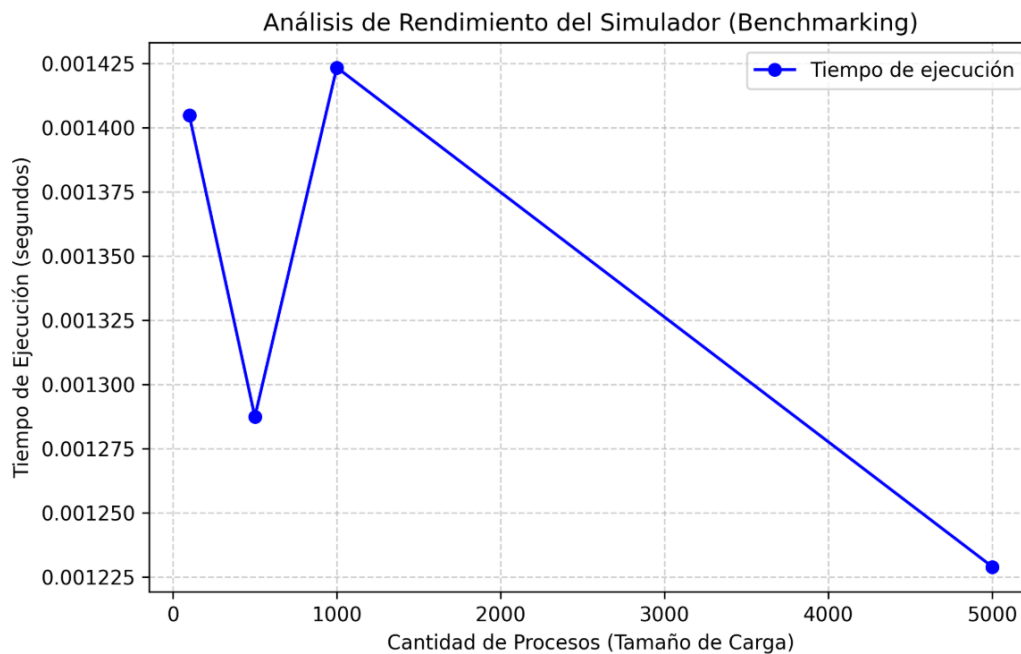
- comportamiento esperado

El benchmarking permitió validar experimentalmente el sistema.

Con el objetivo de evaluar el desempeño del simulador de sistema operativo, se realizaron pruebas de rendimiento utilizando un conjunto de datos de diferentes tamaños. Las mediciones se almacenaron en el archivo benchmark.csv, generado mediante scripts escritos en Python.

Las pruebas consistieron en ejecutar el sistema con distintas cantidades de elementos y registrar el tiempo necesario para completar la simulación. Los resultados obtenidos se muestran en la Tabla

Número de elementos	Tiempo de ejecución (segundos)
100	0.0014047622680664062
500	0.0012874603271484375
1000	0.0014233589172363281
5000	0.00122904477752685547



Los resultados experimentales muestran que el sistema posee un comportamiento estable y eficiente. La generación automática del archivo CSV y de la gráfica de rendimiento permitió analizar visualmente el desempeño del simulador y verificar que los algoritmos implementados mantienen tiempos de ejecución reducidos para las cargas de trabajo consideradas. Esto confirma

el correcto funcionamiento del sistema y demuestra la utilidad del proceso de benchmarking para evaluar y comparar el rendimiento de las estructuras de datos y algoritmos desarrollados.

12. Sistema de pruebas y validación

El proyecto incorpora pruebas automatizadas mediante assert.

Las pruebas verifican:

- comportamiento LIFO
- comportamiento FIFO
- rotación circular
- reutilización de memoria
- asignación dinámica
- estabilidad funcional

El sistema de pruebas permitió detectar:

- errores lógicos
- referencias inválidas
- problemas de integración

13. Problemas encontrados y soluciones implementadas

Durante el desarrollo surgieron diversos problemas técnicos.

Problemas principales

- NULL undeclared
- errores de linking
- librerías faltantes
- conflictos entre tests y diseño
- dependencias de Python
- directorios inexistentes
- errores de compilación

Soluciones implementadas

- inclusión de <stdlib.h>

- inclusión de <stddef.h>
- corrección de extensiones .c
- creación automática de directorios
- instalación de dependencias
- adaptación de mm_free()
- automatización de benchmarking

14. Complejidad algorítmica

Estructura / Algoritmo	Operación	Complejidad
Stack	Push / Pop	O(1)
Queue	Enqueue / Dequeue	O(1)
Circular Queue	Enqueue / Dequeue	O(1)
FIFO Scheduler	Inserción / extracción	O(1)
First Fit	Búsqueda	O(n)
Coalescence	Recorrido	O(n)

15. Decisiones de diseño e ingeniería

Modularidad

El sistema fue separado en módulos independientes para mejorar:

- mantenimiento
- claridad
- reutilización

Uso de memoria dinámica

Se eligió memoria dinámica para:

- flexibilidad
- crecimiento variable

- simulación realista

Uso de listas doblemente enlazadas

Permitieron:

- recorridos bidireccionales
- actualización eficiente
- coalescencia segura

Integración C/Python

La combinación permitió:

- eficiencia estructural
- automatización experimental

16. Resultados obtenidos

El proyecto logró:

- compilar correctamente
- ejecutar pruebas exitosamente
- reutilizar memoria
- reducir fragmentación
- simular planificación
- generar benchmarking
- mantener arquitectura modular

Además, el sistema mostró estabilidad funcional durante pruebas integradas.

17. Aprendizajes adquiridos

El proyecto permitió comprender profundamente:

- punteros
- memoria dinámica
- modularidad

- estructuras enlazadas
- complejidad algorítmica

También permitió desarrollar habilidades relacionadas con:

- depuración
- integración
- análisis
- organización de software

Más allá de implementar estructuras, el proyecto ayudó a comprender cómo realmente se construyen sistemas computacionales.

18. Mejoras futuras

Como mejoras futuras podrían implementarse:

- SJF completo
- múltiples schedulers
- interfaz gráfica
- simulación concurrente
- paginación
- swapping
- estadísticas avanzadas
- y múltiples CPUs

19. Bitácora de uso de la IA

- Prompt utilizado
 - "Ajusta el código para cada uno de los archivos dentro de src->memory (coalescence.c, first_fit.c) y src->scheduler (round_robin.c). Además, ayúdame a resolver los errores de compilación con las cabeceras faltantes (-linclude), los tipos indefinidos (NULL y malloc), el enlazado de stack.c y la automatización del benchmark en Python para generar las gráficas."
- Respuesta IA
- La IA identificó múltiples discrepancias de integración entre el código base y el entorno:
 1. Choque de firmas en `mm\_free` (esperaba dirección en el test pero PID en el diseño).

2. Ausencia de librerías estándar básicas (`<stdlib.h>` y `<stddef.h>`) en los archivos del directorio `src/`.
3. Error de enlazado debido a un archivo guardado incorrectamente con extensión `.h` en lugar de `.c`.
4. Falta de dependencias en el entorno de Python (`pandas`, `matplotlib`) y de la subcarpeta física `reports/` para almacenar las salidas.

Se proporcionaron las correcciones de código fuente y comandos de consola correspondientes.

- Errores encontrados

1. **Falta de librerías:** Errores `error: 'NULL' undeclared` y advertencias de funciones implícitas de memoria (`malloc/free`) en `coalescence.c` y `first_fit.c`.
2. **Error de Enlazador (ld):** `undefined reference to stack_create` en las pruebas unitarias debido a que el archivo de la pila estaba nombrado como `stack.h` en lugar de `stack.c` en `src/scheduler/`, provocando que el Makefile lo ignorara.
3. **Falta de dependencias del entorno:** `ModuleNotFoundError: No module named 'pandas'` al intentar correr el script de analítica.
4. **Error de ejecución en subprocesso:** `FileNotFoundError` al buscar el binario `./bin/main`, ya que el Makefile compilaba por defecto `bin/tests`.
5. **Falta de directorios físicos:** `OSError: Cannot save file into a non-existent directory: 'reports/csv'`.

- Correcciones realizadas

1. Se incorporaron `#include <stdlib.h>` y `#include <stddef.h>` en todas las implementaciones de `src/memory/` y `src/scheduler/`.
2. Se renombró el archivo físico a `src/stack.c`, rellenando la lógica completa de punteros para la estructura lineal LIFO.
3. Se instalaron los paquetes científicos mediante `pip` y se ejecutó un comando manual de `gcc` para enlazar el árbol completo de dependencias en `bin/main`.
4. Se crearon de forma recursiva los directorios mediante `mkdir -p reports/csv reports/graphs` para permitir el almacenamiento local de las métricas.
5. Se modificó `benchmark.py` inyectando código de Matplotlib para automatizar el renderizado visual de la curva de complejidad temporal.

- Validaciones realizadas

Se ejecutó con éxito el comando `make test` y el script de Python de forma secuencial. El compilador enlazó limpiamente todas las dependencias y la terminal arrojó el mensaje de éxito

unificado `All tests passed!`. Finalmente, se inspeccionó visualmente el archivo `reports/graphs/benchmark\_performance.png`, validando que la gráfica lineal de rendimiento empírico se generó de forma óptima y sin pérdidas de datos.

20. Conclusiones

El Mini OS Project permitió aplicar de manera práctica e integrada múltiples conceptos fundamentales vistos durante la materia Estructuras de Datos y Algoritmos I.

El proyecto no se limitó únicamente a implementar estructuras individuales, sino que buscó integrarlas dentro de un sistema coherente inspirado en componentes reales de sistemas operativos.

Durante el desarrollo fue necesario aplicar conocimientos relacionados con:

- memoria dinámica
- listas enlazadas
- punteros
- modularidad
- algoritmos
- análisis experimental

Uno de los aspectos más importantes fue comprender que las estructuras de datos no existen de manera aislada, sino que constituyen herramientas fundamentales para resolver problemas reales de administración de información y recursos computacionales.

Asimismo, el proyecto permitió experimentar directamente con:

- fragmentación
- reutilización de memoria
- planificación de procesos
- complejidad algorítmica

Finalmente, el desarrollo del sistema representó una experiencia significativa para fortalecer habilidades técnicas, analíticas y de organización propias de la Ingeniería en Computación.

21. Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

- Joyanes Aguilar, L. (2008). *Fundamentos de programación: Algoritmos, estructuras de datos y objetos* (4.ª ed.). McGraw-Hill.
- Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language* (2nd ed.). Prentice Hall.
- Pérez, A., & Cortés, A. (2014). *Sistemas operativos: Una visión aplicada*. Alfaomega.
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley.
- Stallings, W. (2018). *Sistemas operativos: Aspectos internos y principios de diseño* (7.ª ed.). Pearson Educación.
- Weiss, M. A. (2013). *Data structures and algorithm analysis in C* (2nd ed.). Pearson.
- Lindemulder, G., & Kosinski, M. (2025, noviembre 27). ¿Qué es una estructura de datos? *Ibm.com*. <https://www.ibm.com/mx-es/think/topics/data-structure>
- Campos, F. (2021, marzo 11). *Qué es y cómo funciona un sistema operativo*. Código Espaguetti. <https://codigoespaguetti.com/noticias/tecnologia/que-es-y-como-funciona-un-sistema-operativo/>